Project Exercises

Project Exercise 1 :- Blog Website

- Topic Covered :- HTML
- Description :- Build a simple blog webpage that includes essential HTML elements like headings, paragraphs, unordered and ordered lists, links, and images. The page should start with a DOCTYPE declaration and include the `<html>`, `<head>`, and `<body>` tags
- Skills Covered :- HTML structure and embedding media.

Project Exercise 2 :- Contact Us Form

- **Topic Covered:** HTML
- **Description:** Create a "Contact Us" form that includes input fields for name, email, phone number, and a message. The form should have a submit button, and the fields should have basic validation, such as ensuring the name, email, and phone number fields are required before submission. Use appropriate input types and attributes for each field to ensure the form is user-friendly and accessible.
- **Skills Covered:** Form creation, input elements, form validation, attributes.

Project Exercise 3: Styling The Page

- **Topic Covered:** CSS.
- **Description:** Style the blog page created in Project Exercise 1 by applying CSS. Add background colors to sections, set text colors, and choose appropriate fonts. Use CSS properties to adjust text sizes, line heights, and font styles. Implement padding and margins to create spacing between elements, ensuring the layout is visually appealing and easy to read. Utilize CSS selectors and the box model to enhance the overall presentation of the page.
- **Skills Covered:** CSS styling, applying selectors, text formatting, using padding/margins, and understanding the box model for layout control.

Project Exercise 4: Two-Column Layout with Flexbox

- **Topic Covered:** HTML, CSS
- **Description:** Design a two-column webpage layout using Flexbox. The page should include a header at the top, a sidebar on the left with a list of links, and a main content area on the right. The sidebar should be narrow, and the main content area should take up the remaining space.
- **Skills Covered:** Flexbox for layout management, creating a two-column structure, CSS positioning

---

Project Exercise 5: Styling the Form

- **Topic Covered:** HTML, CSS
- **Description:** Enhance the "Contact Us" form created in Project Exercise 2 by applying CSS for better styling. Add hover effects to the submit button to improve user interaction, change the borders of input fields when they are focused, and refine the overall form layout with proper spacing and alignment. Use properties like `border-radius` for rounded corners, padding for inner spacing, and pseudo-classes like `:hover` and `:focus` for interactivity.
- **Skills Covered:** Advanced form styling, pseudo-classes (`:hover`, `:focus`), border-radius for rounded corners, and using padding to improve form aesthetics.

Project Exercise 6: Responsive Photo Gallery with CSS Grid

- **Topic Covered:** HTML, CSS
- **Description:** Design a responsive photo gallery using CSS Grid. The gallery should display images of varying sizes in a grid layout, where the number of columns adjusts based on the screen size. Use media queries to ensure the layout is responsive and looks great on different devices, such as desktops, tablets, and mobiles. The gallery should adapt to different screen widths, adjusting the grid to display images in an optimal manner.
- **Skills Covered:** CSS Grid for layout management, media queries for responsiveness, and creating flexible, adaptive designs.

Project Exercise 7: Change Background Color with JavaScript

- **Topic Covered:** JavaScript

- **Description:** Write a JavaScript script that changes the background color of the webpage when a button is clicked. The button should trigger a function that alters the `background-color` of the webpage. Use DOM manipulation to select the button element and apply an event listener for the click event.
- **Skills Covered:** Basic JavaScript syntax, functions, event handling, and DOM manipulation.

Project Exercise 8: Dynamically Add List Items

- **Topic Covered:** HTML, CSS, JavaScript, DOM
- **Description:** Create a webpage with an unordered list and a button. When the button is clicked, use JavaScript to dynamically add new list items to the unordered list. The script should create a new list item element and append it to the list each time the button is clicked. Style the list and button using CSS for better presentation. Focus on DOM manipulation techniques like `document.createElement()` and `appendChild()` to achieve this functionality.
- **Skills Covered:** DOM manipulation, `document.createElement()`, `appendChild()`, event listeners, and DOM traversal for interacting with elements.

Project Exercise 9: Simple Calculator

- **Topic Covered**: HTML, CSS, JavaScript
- **Description:** Build a simple calculator using HTML, CSS, and JavaScript. The calculator should have buttons for numbers (0-9), basic operators (+, -, *, /), and a display area to show the input and result. Use JavaScript to handle button clicks, perform calculations, and update the display. Style the calculator using CSS for a clean, user-friendly interface.
- **Skills Covered:** Creating forms and buttons, handling user input with JavaScript, basic arithmetic operations, DOM manipulation, and CSS styling for layout and responsiveness.

Project Exercise 10: JavaScript Form Validation

- **Topic Covered:** JavaScript, HTML
- **Description:** Create a form that includes fields for name, email, and password. Write a JavaScript function to validate the form before

submission. Ensure that the name field is not empty, the email follows a valid email format, and the password meets certain criteria (e.g., minimum length). If the validation fails, display an error message next to the respective field and prevent the form from being submitted. Style the form and error messages for clarity.

- **Skills Covered:** JavaScript form validation, regular expressions for email validation, handling form events, DOM manipulation, and displaying dynamic error messages.

Project Exercise 11: To-Do List App with Event Handling

- **Topic Covered:** JavaScript, DOM
- **Description:** Build a simple to-do list application where users can add new tasks, mark them as complete, and remove tasks. Use JavaScript event listeners to handle interactions such as adding tasks when the user submits input, toggling task completion when clicked, and deleting tasks when a remove button is clicked. Dynamically update the DOM based on user actions to reflect the changes in the to-do list.
- **Skills Covered:** Event listeners, dynamic DOM manipulation, user interaction, and managing UI updates based on actions.

Project Exercise 12: Button Animation with CSS Transitions

- **Topic Covered:** CSS. javascript (optional)
- **Description:** Create a button that animates when clicked. Use CSS transitions to make the button grow when hovered over and shrink when clicked, providing smooth effects. Implement the animation using `@keyframes` and transition properties such as `transform` and `scale`. Ensure the animation is smooth and feels interactive for the user, with appropriate timing and easing functions.
- **Skills Covered:** CSS transitions, animations with `@keyframes`, transition properties like `transform` and `scale`, and smooth interactive effects.

Project Exercise 13: JavaScript Object - Car Information

- **Topic Covered**: JavaScript.

- **Description:** Create a JavaScript object representing a car with properties such as `make`, `model`, and `year`. Display these properties dynamically in the browser by using `document.write()` or manipulating the DOM. Additionally, log the object to the console to showcase how to access and display object properties.
- **Skills Covered:** Working with JavaScript objects, understanding properties, methods, and how to log information to the console and display it on the web page.

Project Exercise 14: Interactive Quiz App

- **Topic Covered:** HTML, CSS, JavaScript, DOM
- **Description:** Create a simple interactive quiz app that presents multiple-choice questions to the user. The user selects answers and submits the quiz. After submission, display the total score based on correct answers. Use JavaScript to handle user interactions, such as selecting answers and calculating the score. Manipulate the DOM to dynamically update the question and show the final score after submission.
- **Skills Covered:** DOM manipulation, event handling, JavaScript functions, conditionals for checking answers, and updating content dynamically.

Project Exercise 15: Responsive Navbar

- **Topic Covered:** HTML, CSS
- **Description:** Design a responsive navigation bar that displays horizontal links on larger screens. When the screen size is reduced (e.g., mobile view), the navbar should transform into a hamburger menu. Use media queries to make the navbar responsive and flexbox for layout management. Ensure the hamburger menu is clickable, and when clicked, it displays the navigation links in a dropdown or slide-in style.
- **Skills Covered:** Media queries for responsiveness, flexbox for layout, positioning for menu items, and implementing responsive design techniques.

Project Exercise 16: Image Slider

- **Topic Covered:** HTML, CSS, JavaScript, DOM.

- **Description:** Build an image slider that automatically cycles through a series of images every few seconds. Add next and previous buttons that allow users to manually navigate between images. Use JavaScript `intervals` to control the automatic image transition, and implement event listeners for the next/previous buttons. Apply CSS transitions for smooth image sliding effects. The slider should be responsive, adjusting to different screen sizes.
- **Skills Covered:** JavaScript intervals for automatic transitions, event listeners for navigation buttons, DOM manipulation for dynamic updates, and CSS transitions for smooth animations.

Project Exercise 17: Countdown Timer

- **Topic Covered:** JavaScript, DOM
- **Description:** Create a countdown timer that starts from a specified time (e.g., 10 minutes) and counts down to zero. The timer should update every second and display the remaining time on the webpage. When the timer reaches zero, an alert should appear notifying the user that time is up. Use `setInterval()` to update the timer every second and `clearInterval()` to stop the countdown when it reaches zero.
- **Skills Covered:** `setInterval()` for periodic updates, `clearInterval()` for stopping the timer, time manipulation, and updating the DOM to display the countdown.

Project Exercise 18: Custom Modal Popup

- **Topic Covered:** HTML, CSS, JavaScript
- **Description:** Create a custom `modal` popup that appears when a button is clicked. The modal should display a message or content and include a "close" button that hides the modal when clicked. Use JavaScript for event handling, such as showing the modal when the button is clicked and hiding it when the close button is clicked. Style the modal using CSS to ensure it is positioned correctly on the screen and is initially hidden.
- **Skills Covered:** DOM manipulation for showing and hiding the modal, event handling for button clicks, CSS positioning for the modal, and controlling visibility with CSS and JavaScript.

Project Exercise 19: Local Storage To-Do App

- **Topic Covered:** HTML, CSS, JavaScript
- **Description:** Build a to-do list application that allows users to add, delete, and mark tasks as completed. The tasks should be stored in the browser's local storage, ensuring they persist even after the page is reloaded. Use JavaScript to handle the creation and deletion of tasks, update the DOM accordingly, and utilize the `localStorage` API to save tasks. When the page is refreshed, the tasks should be reloaded from local storage and displayed.
- **Skills Covered:** Using `localStorage` for persistent data storage, DOM manipulation to dynamically update the task list, and handling user input with event listeners.

Project Exercise 20: Interactive Weather App

- **Topic Covered:** HTML, CSS, JavaScript, DOM
- **Description:** Build a weather app that fetches real-time weather data from a public API (such as OpenWeatherMap) and displays it in a user-friendly interface. The app should allow users to search for the weather by city name. Use the Fetch API to request data, handle the JSON response, and update the DOM with the weather information (e.g., temperature, humidity, and conditions). Include error handling for invalid city names or network issues.
- **Skills Covered:** Using the Fetch API to retrieve data, handling JSON responses, DOM manipulation to display the weather data, and implementing error handling for API requests.

Project Exercise 21: Hello React - Your First React App

- **Topic Covered:** React
- **Description:** Set up a React development environment using `Create React App` or `Vite`. Build your first app that displays “Hello, React!” and learn about JSX by modifying the greeting message. Explore the Real DOM vs. `Virtual DOM`, install an additional package (e.g., react-icons), and understand the folder structure of a React project.
- **Skills Covered:** React setup, JSX syntax, Virtual DOM vs. Real DOM, NPM package usage, React project folder structure.

Project Exercise 22: Movie Card Styling in React

- **Topic Covered:** React, CSS, TailwindCSS, Animation
- **Description:** Create a movie card component in React that displays movie information such as title, image, description, and rating. Style the card using inline styles and CSS modules. Integrate `TailwindCSS` to style elements like buttons and headers, and apply responsive design with media queries. Use conditional styling based on the `movie's rating` (e.g., change color or style for high vs. low ratings). Add an animated hover effect to the movie card using Framer Motion or GSAP for enhanced interactivity.
- **Skills Covered:** Inline styles, CSS modules, TailwindCSS, conditional styling, media queries, responsive design, animations with Framer Motion/GSAP.

Project Exercise 23: Movie List with State and Props

- **Topic Covered:** React, State, Props
- **Description:** Create a functional component that receives data via props and displays it. Use the `useState` hook to manage and update the component's state dynamically. Render a list of movies or items using the `map()` method and allow for updates to the list through state changes. Create parameterized components that accept dynamic props, and implement event handling for interactions like button clicks or form submissions.
- **Skills Covered:** State management with `useState`, passing data via props, rendering dynamic lists with `map()`, parameterized components, event handling in React.

Project Exercise 24: React Memory Game with Component Lifecycle

- **Topic Covered:** React, Component Lifecycle, Hooks
- **Description:** Build a memory game app using React where users match pairs of cards. Start by creating a class component and implement the `componentDidMount` method to initialize the game state. Then, refactor the component into a functional one and use the `useEffect` hook to handle side effects like shuffling the cards or checking for matches. Implement the mounting, updating, and unmounting of components when the game starts,

and use `useContext` to share the game state (e.g., score, matched cards) across the app.

- **Skills Covered:** Component lifecycle, `componentDidMount`, `useEffect` hook, managing side effects, `useContext` for global state management, building interactive memory game logic.

Project Exercise 25: Contact Card App with React Hooks

- **Topic Covered:** React, useState, useEffect, useContext, useMemo, useCallback, useRef, Custom Hooks.
- **Description:** Build a contact card app where users can enter contact details (name, phone number, email) into a form. When the user submits the form, the contact is added to a list and displayed on the screen. Use `useState` to manage the form inputs, `useContext` to share the list of contacts globally, and `useEffect` to manage any side effects (e.g., resetting the form after submission). Optimize performance with `useMemo` and `useCallback`. Use `useRef` to focus on the name input field when the form is loaded. Create a custom hook to manage form validation or fetch data if required.
- **Skills Covered:** `useState`, `useEffect`, `useContext`, `useMemo`, `useCallback`, `useRef`, creating custom hooks, managing global state with context, form handling, and dynamic list rendering.

Project Exercise 26: Multi-Page Authentication App with React Router

- **Topic Covered:** React Router, Routing, useNavigate, LocalStorage.
- **Description:** Build a multi-page website with React Router that includes pages for `Register`, `Login`, and `Home`. Set up `react-router-dom` to navigate between these pages. Use a navigation bar (`Navbar`) for easy navigation. In the `Register` and `Login` pages, create forms for users to input their data, and validate the form data before submission. If the form data is correct, store the user's information in `localStorage` for authentication and navigate to the `Home` page. Implement basic form validation (e.g., check for valid email or password). Use `useNavigate` for programmatic navigation and handle conditional rendering based on whether the user is authenticated.
- **Skills Covered:** React Router setup, `<Link>` for navigation, `useNavigate` for programmatic navigation, form validation, localStorage for authentication,

creating and handling dynamic routes, conditional rendering based on authentication state.

Project Exercise 27: Money Management App with Redux

- **Topic Covered:** Redux, React-Redux, Redux Toolkit, Redux Thunk, State Management.
- **Description:** Build a money management application using Redux for state management. The app should allow users to perform credit and debit actions and maintain a transaction history with details such as time, date, and transaction type (credit/debit). Implement Redux in the app, creating a store, actions, and reducers to handle the credit and debit functionalities. Use `react-redux` to connect the Redux store to your React components. Optimize the Redux logic with Redux Toolkit for more efficient state management. Additionally, implement async actions using Redux Thunk to handle time-dependent actions (e.g., saving transaction details to an external database).
- **Skills Covered:** Redux state management, `react-redux` for connecting components to the store, Redux actions, reducers, Redux Toolkit for simplified logic, async actions with Redux Thunk, transaction history management.

Project Exercise 28: Dynamic Login Form with Validation

- **Topic Covered:** React, Dynamic Forms, Validation, axios, Fetch API
- **Description:** Build a dynamic login form in React with fields for email and password. Implement two-way data binding using React's `useState` for the form inputs. Validate the inputs on the client side (e.g., check for a valid email format and ensure the password is not empty). On form submission, use `axios` or `fetch` to send the form data to a mock API endpoint. Display user feedback such as loading, success, or error based on the response from the mock API.
- **Skills Covered:** Two-way binding, client-side form validation, form submission with `axios` or `fetch`, managing form state with `useState`, displaying feedback (loading, success, error).

Project Exercise 29: Optimizing TMDB Application Performance

- **Topic Covered:** React Performance, Code Splitting, Memoization, TMDB API, Chrome DevTools.
- **Description:** Build a movie discovery app using the free TMDB (The Movie Database) API and focus on optimizing its performance. Implement code splitting using `React.lazy` and `Suspense` to load components like movie lists and details only when needed, improving the app's initial load time. Use `React.memo` to prevent unnecessary re-renders of movie components. Cache expensive calculations, such as movie search results, with `useMemo` and `useCallback` to improve rendering efficiency. Use `axios` or `fetch` to make API calls to TMDB and display movie details dynamically. Analyze the performance of the app using Chrome DevTools and Lighthouse, and optimize the component structure for faster rendering.
- **Skills Covered:**
  - Code splitting with `React.lazy` and `Suspense`
  - Performance optimization with `React.memo`, `useMemo`, and `useCallback`
  - Fetching data from TMDB API
  - Performance analysis using Chrome DevTools and Lighthouse
  - Optimizing rendering efficiency in React components

Project Exercise 30: Building a `Spotify` -Like Song Streaming Application

- **Topic Covered:** Full-Stack React, React Router, Redux/Context API, API Integration, TailwindCSS, Performance Optimization, Deployment.
- **Description:** Build a complete song streaming application similar to `Spotify` using React. Store all song files in the public folder and provide functionalities like playing songs, pausing, skipping, and displaying song information. Implement routing with React Router to navigate between pages like the home page, song details, and playlists. Use `Redux` or Context API to manage global state for things like the current song, playlist, and user preferences. Integrate a mock API to fetch song metadata, and use `TailwindCSS` for a responsive and modern user interface. Optimize the app's `performance` for smooth playback and deploy it to a hosting platform like `Netlify` or `Vercel`.
- **Skills Covered:**

- Full-stack React development
- React Router for navigation
- State management with `Redux` or `Context API`
- API integration for song metadata
- Responsive design with TailwindCSS and media queries
- `Performance` `optimization` for media handling
- Deployment to platforms like Netlify or Vercel

Project Exercise 31: Hello World Backend

- **Topic Covered:** Running Node.js Script, CommonJS vs ES6 Modules.
- **Description:** Create a basic Node.js script that displays "Namaste Duniya" in the terminal. Write the script using both CommonJS (`require`) and ES6 module (`import`) syntax to understand their differences.
- **Skills Covered:**
  - Node.js fundamentals
  - Understanding CommonJS and ES6 module syntax
  - Running Node.js scripts in the terminal

Project Exercise 32: Basic File System Explorer

- **Topic Covered:** Core Modules (os, fs, path).
- **Description:** Build a Node.js script that explores the file system. The script should read and display all files and subdirectories within a specified directory using `fs` and `path` modules. Include the ability to handle errors (e.g., if the directory doesn't exist).
- **Skills Covered:**
  - File system operations using `fs`
  - Manipulating paths with `path`
  - Error handling in Node.js
  - Using core modules effectively

Project Exercise 33: Simple HTTP Server

- **Topic Covered:** Creating a Server with Node.js.
- **Description:** Create a basic HTTP server using Node.js. The server should respond with "Hello, Browser!". Handle different routes and return appropriate status codes (e.g., 404 for unknown routes).
- **Skills Covered:**
  - Building HTTP servers with Node.js
  - Serving plain text and HTML responses
  - Understanding and implementing HTTP status codes
  - Routing and basic request handling

Project Exercise 34: Routing in HTTP Servers

- **Topic Covered:** Routing and Status Codes.
- **Description:** Build an HTTP server in Node.js that handles routes for `/home`, `/about`, and a fallback route `/404`. Serve appropriate HTML content for each route and ensure the use of proper HTTP status codes (e.g., 200 for successful routes and 404 for missing routes).
- **Skills Covered:**
  - Implementing routing in HTTP servers
  - Handling HTTP status codes
  - Serving static HTML content
  - Error handling for undefined routes

Project Exercise 35: Express.js Blog

- **Topic Covered:** Express.js Basics
- **Description:** Build a simple blog backend using Express.js. Implement CRUD (Create, Read, Update, Delete) functionality with endpoints like `/add-post`, `/view-post/:id`, `/update-post/:id`, and `/delete-post/:id`. Use an in-memory array to store blog posts for simplicity.
- **Skills Covered:**
  - Setting up an Express server

- Creating and managing RESTful routes
- Handling HTTP methods (GET, POST, PUT, DELETE)
- Managing request parameters and body parsing

Project Exercise 36: Serving Static Files

- **Topic Covered:** Express.js Static Middleware.
- **Description:** Use Express.js to serve a static folder containing assets like CSS, images, and JavaScript files. Create a basic homepage that uses these assets for styling and functionality. Ensure a proper folder structure for better organization.
- **Skills Covered:**
  - Setting up and using `express.static` middleware
  - Serving static assets (CSS, images, JS files)
  - Organizing folder structures for web projects
  - Building a simple homepage to utilize static files

Project Exercise 37: Dynamic Website with EJS

- **Topic Covered:** Template Engine (`EJS`)
- **Description:** Create a dynamic webpage using `EJS` to display a list of user profiles. Add functionality to accept new profiles through a form and dynamically render the updated list.
- **Skills Covered:** EJS syntax, rendering dynamic data, working with forms.

Project Exercise 38: Middleware in Action

- **Topic Covered:** `Middleware` in Express.js
- **Description:** Create an Express.js application that uses middleware to log request details (`URL`, `HTTP` `method`, and `timestamp`) for every request. Implement custom error-handling middleware to return a user-friendly message for invalid routes or errors.
- **Skills Covered:**
  - `Application-level` middleware

- Request `logging` and `debugging`
- Error handling in `Express.js`.

Project Exercise 39: File Upload System

- **Topic Covered:** Handling Files with `Multer`.
- **Description:** Create an Express.js backend that allows users to upload profile pictures. Use Multer for handling file uploads, `validate` file types (e.g., allow only `images`), and store them on the `disk` or in memory.
- **Skills Covered:**
  - File upload handling with Multer.
  - Validating file `types` and `sizes`.
  - Using memory and disk `storage` options for uploads.

Project Exercise 40: MongoDB Setup

- **Topic Covered:** `MongoDB` `Basics`.
- **Description:** Set up a MongoDB database (locally or on a cloud service like `MongoDB` `Atlas`). Use Mongoose to define a schema for a “books” collection with fields such as title, author, and genre. Connect your Node.js application to the database and add functionality to `insert` and `retrieve` book data.
- **Skills Covered:** Setting up MongoDB locally or on the cloud , Defining Mongoose schemas and models, Connecting Node.js to MongoDB.

Project Exercise 41: REST API for Library Management

- **Topic Covered:** `REST API` Development.
- **Description:** Build a REST API for a library system using `Express.js` and MongoDB. Implement endpoints to perform `CRUD` operations:
  - Add new books (`POST` /books).
  - View all books or a specific book (`GET` /books or GET /books/:id).
  - Update book details (`PUT` /books/:id).
  - Delete a book (`DELETE` /books/:id).

Test the API endpoints using Postman to ensure proper functionality.

- **Skills Covered:** Designing and implementing `RESTful APIs` .

Project Exercise 42: Optimizing Search with Database Indexing

- **Topic Covered:** MongoDB `Indexing` .
- **Description:** Enhance your library management database by adding indexes to `frequently` `queried` fields like genre and author. Perform search operations using these indexed fields to demonstrate faster query results. Experiment with compound indexes (e.g., `combining` `genre` and `author`) and observe their impact on query `performance` .
- **Skills Covered:**
  - Basics of indexing in MongoDB.
  - Creating single-field and compound indexes.
  - Optimizing and analyzing database queries.

Project Exercise 43: Securing Users with Authentication

- **Topic Covered:** Authentication and `Authorization` .
- **Description:** Build a user `authentication` system where users can register and `log` in. Securely hash passwords using bcrypt during registration, and validate credentials during login. On successful login, generate and return a JWT (JSON Web Token) to the user. Use the JWT for `session` management and secure `protected` routes.
- **Skills Covered:**
  - `Password` hashing with `bcrypt` .
  - `JWT` creation and validation.
  - Securing user data and routes.
  - Authentication and `authorization` basics.

Project Exercise 44: Real-Time Chat Room

- **Topic Covered:** `WebSockets` and `Socket.io.`

- **Description:** Create a real-time chat application that allows users to join specific chat rooms and exchange messages with other participants. Implement features like user `connection` / `disconnection` notifications and chat room management.
- **Skills Covered:**
  - WebSocket communication with Socket.io
  - Handling real-time events (e.g., message `broadcasting`, user activity)
  - Managing multiple chat `rooms`
  - Event-driven architecture in Node.js

Project Exercise 45: Library API with Redis Caching

- **Topic Covered:** `Redis Caching`.
- **Description:** Optimize your library management API by implementing Redis caching for `frequently` requested data, such as the book list. Ensure the cache is invalidated whenever the data is updated to maintain `consistency`.
- **Skills Covered:**
  - Setting up and connecting to Redis.
  - Implementing caching mechanisms.
  - Improving API response time.
  - Cache invalidation strategies.

Project Exercise 46: Centralized API Error Handler

- **Topic Covered:** Error Handling in Express.
- **Description:** Create a `utility` `class` for handling and standardizing errors across your API. Implement custom error-handling `middleware` in Express and use `next()` to pass errors through the middleware chain.
- **Skills Covered:**
  - `Centralized` error management.
  - Building `custom` error classes.
  - Express middleware for `error` `handling`.

- Debugging and improving API `reliability`.

Project Exercise 47: Build an AI-powered Resume Reviewer

- **Topic Covered:** Generative AI for Applications
- **Description:** Create a web application where users upload their resumes, and AI provides feedback for improvement. Use `OpenAI` APIs (or `Gemini` tools) for generating suggestions.
- **Skills Covered:** Working with AI APIs, file handling, NLP basics.

Project Exercise 48: Social Media Post Automation

- **Topic Covered:** Social Media Automation with Generative AI
- **Description:** Build a tool that generates engaging captions for Instagram posts based on an input image or text.
- **Skills Covered:** Integrating Generative AI, understanding image-to-text models.

Project Exercise 49: Virtual Interview Assistant

- **Topic Covered:** LangChain for Generative AI Applications
- **Description:** Develop a chatbot for virtual interviews using `LangChain`. The bot should ask questions based on a job profile and provide feedback on answers.
- **Skills Covered:** LangChain basics, conversational AI, integration with APIs.

Project Exercise 50: Create a Basic PWA

- **Topic Covered:** Service `Workers` and `Manifest.json`
- **Description:** Convert a simple website into a `PWA`. Add a manifest file and implement a service worker for `offline functionality`.
- **Skills Covered:** PWA fundamentals, service worker lifecycle, manifest setup.

Project Exercise 51: Implement Lazy Loading and Code Splitting

- **Topic Covered:** Performance `Optimization` in PWAs

- **Description:** Create a PWA that `lazy` `loads` images and splits code into smaller chunks for faster load times.
- **Skills Covered:** Lazy loading, code splitting, optimizing `asset` `delivery`.

Project Exercise 52: Advanced Caching with Service Workers

- **Topic Covered:** Caching Strategies in `PWAs`
- **Description:** Build a news app that caches articles and serves them offline. Use strategies like `Cache First` and `Network First`.
- **Skills Covered:** Advanced caching, offline functionality, debugging with DevTools.

Project Exercise 53: Build and Run a Docker Container

- **Topic Covered:** Docker Basics
- **Description:** Write a Dockerfile to containerize a simple Node.js application. Build a Docker image from the Dockerfile and run the application inside a Docker container. Test the app's functionality from within the container.
- **Skills Covered:** Writing Dockerfiles, containerization, Docker CLI.

Project Exercise 54: Orchestrate with Kubernetes

- **Topic Covered:** `Kubernetes` Basics.
- **Description:** Deploy a Node.js app with a MongoDB database using Kubernetes. Create `YAML` files for `pods`, `services`, and `deployments`. Use kubectl to manage the application, ensuring communication between the Node.js app and the database via a Kubernetes `service`.
  - **Skills Covered:**
  - Understanding Kubernetes architecture (Pods, Services, Deployments).
  - Writing YAML configuration for Kubernetes.
  - Managing resources using kubectl commands.
  - Service orchestration for multi-container apps.

Project Exercise 55: Automate with CI/CD

- **Topic Covered:** Automating Deployments.
- **Description:** Build a `CI/CD` pipeline using GitHub Actions to automate the building and deployment of a Docker container. Use `Terraform` to define and provision infrastructure, such as servers or cloud services, required for `deployment`.
- **Skills Covered:**
  - Setting up CI/CD pipelines with GitHub Actions
  - Writing `Terraform` scripts for infrastructure as code
  - Automating `Docker` container builds and deployments
  - Integrating infrastructure `automation` with deployment processes

---

Project Exercise 56: Build Your First Microservice

- **Topic Covered:** Microservice Architecture.
- **Description:** Create a simple `microservice` using Node.js and Express. The microservice will handle a specific function, such as managing a “Product” entity, with endpoints for `adding`, `retrieving`, `updating`, and `deleting` products. Use MongoDB as the database.
- **Skills Covered:**
  - Building microservices with Node.js and Express.
  - Designing `RESTful` APIs
  - `CRUD` operations with MongoDB
  - `Modular` application structure for microservices

Project Exercise 57: Build an API Gateway with Event-Driven Communication

- **Topic Covered:** API Gateway for `Microservices` & `Redis Pub/Sub.`
- **Description:** Create an API Gateway using Express.js to route requests to multiple microservices. Implement `event-driven` communication between these microservices using Redis Pub/Sub. For example, the API Gateway routes requests to an Order service, which then notifies a User service about a new order via `Redis` Pub/Sub. Add features like rate limiting and authentication at the API Gateway level for `security`.

- **Skills Covered:**
  - API Gateway setup with Express.js.
  - Proxying requests and routing.
  - Redis Pub/Sub for inter-service communication.
  - Event-driven architecture.
  - Security features like authentication and rate limiting.

Project Exercise 58: Deploy a Node.js App to AWS

- **Topic Covered:** Cloud Deployment.
- **Description:** Launch an `AWS` `EC2` instance and set up the necessary environment for deploying a Node.js application. Configure `NGINX` as a reverse proxy to handle incoming traffic and link a custom domain to your app for seamless access.
- **Skills Covered:**
  - Setting up AWS EC2 instances.
  - Setting up AWS EC2 instances.
  - Installing and `running` a Node.js app on the cloud.
  - Configuring NGINX as a reverse proxy.
  - Domain masking and `DNS` setup for custom domains.

Project Exercise 59: Deploy with DigitalOcean App Platform

- **Topic Covered:** Simplified Deployment.
- **Description:** `Deploy` a multi-container application using `DigitalOcean's` App Platform. Explore its simplicity by setting up services, and test `scalability` by configuring replicas for your app.
- **Skills Covered:**
  - Deploying applications on DigitalOcean App Platform.
  - Managing multi-container setups.
  - `Scaling` `applications` by adding replicas.
  - `Monitoring` application `performance`.